DEVELOPMENT OF A LATENCY-AWARE LLM FRAMEWORK FOR TASK PLANNING AND AUTONOMOUS NAVIGATION IN ROS 2 MOBILE ROBOTS

Sree Raam and Assoc. Prof. Ir. Dr. Zool Hilmi Ismail

Centre for Artificial Intelligence and Robotics (CAIRO)iKohza CAIRO, Department of Electronic Systems Engineering,
Malaysia-Japan International Institute of Technology
Universiti Teknologi Malaysia
sreeraam@graduate.utm.my

Abstract – Autonomous Mobile Robots (AMRs) have become increasingly important in industrial inspection, surveillance, logistics, and service applications owing to their capability to improve operational efficiency while reducing human intervention. Conventional robotic systems commonly rely on predefined coordinates and manually designed mission sequences, limiting their adaptability to changing environments and operational requirements. Recent advancements in Large Language Models (LLMs) provide opportunities to enhance robotic autonomy through natural language understanding, contextual reasoning, and intelligent task planning.This paper presents the development and evaluation of OpenClaw, a latency-aware framework that integrates LLM-based task planning with autonomous navigation in ROS 2 mobile robots. The proposed framework was implemented using a TurtleBot3 Burger operating in a Gazebo simulation environment and an NVIDIA Jetson Orin Nano as an edge-computing platform. The developed system combines local language model inference through Ollama, LiDAR-based perception, autonomous navigation, safety monitoring, manoeuvre-based recovery behaviours, benchmark automation, and real-time dashboard visualization within a unified software architecture. Three lightweight open-source language models, namely Qwen2.5-3B, Gemma2-2B, and DeepSeek-R1-1.5B, were comparatively evaluated under identical inspection scenarios. A total of 120 autonomous inspection missions comprising forty runs per language model were conducted to assess planning latency, navigation efficiency, mission completion capability, safety compliance, recovery performance, and deployment readiness. The proposed framework demonstrates the feasibility of integrating modern language intelligence into autonomous robotic systems while maintaining reliable operation under edge-computing constraints.

1. Introduction

Autonomous Mobile Robots (AMRs) have emerged as one of the enabling technologies of Industry 4.0 and are increasingly deployed in applications such as warehouse automation, infrastructure inspection, healthcare assistance, surveillance, and service robotics. The growing complexity of these applications requires robotic systems to perform tasks autonomously while maintaining safe and reliable operation. Despite significant advancements in robot perception and navigation, many existing robotic systems still depend on predefined coordinates, manually designed state machines, and hardcoded mission sequences, limiting their adaptability when mission objectives or environmental conditions change.

Robot Operating System 2 (ROS 2) provides a robust middleware framework for autonomous robotic applications through its modular architecture, real-time communication capabilities, and compatibility with modern sensing technologies such as LiDAR, cameras, and odometry systems. However, conventional ROS-based inspection robots often require extensive manual configuration and predefined navigation goals, reducing their capability to understand high-level user intentions and generate adaptive mission plans autonomously.

Recent developments in Artificial Intelligence have introduced Large Language Models (LLMs) capable of natural language understanding, contextual reasoning, and task decomposition. Models such as Qwen, Gemma, and DeepSeek enable users to communicate inspection objectives using intuitive natural language commands instead of manually specifying navigation coordinates or mission sequences. Nevertheless, deploying LLMs in autonomous robotic systems remains challenging due to computational limitations, inference latency, and the requirement to maintain safe navigation behaviour, particularly on resource-constrained edge-computing platforms.

To address these challenges, this study presents OpenClaw, a latency-aware framework for intelligent task planning and autonomous navigation in ROS 2 mobile robots. The proposed framework integrates local language model inference through Ollama, LiDAR-based environmental perception, autonomous navigation, safety monitoring, manoeuvre-based recovery mechanisms, benchmark automation, and dashboard-based visualization within a unified robotic platform. The framework was evaluated using Qwen2.5-3B, Gemma2-2B, and DeepSeek-R1-1.5B through 120 autonomous inspection missions conducted under identical experimental conditions. The findings of this work are expected to provide practical insights into the deployment of lightweight language models for edge-based autonomous robotic systems and contribute towards the development of more intelligent inspection platforms for future industrial applications

Methodology

The proposed OpenClaw framework was developed to enable intelligent inspection missions by integrating Large Language Models (LLMs), autonomous navigation, environmental perception, safety monitoring, and benchmark automation within a unified ROS 2 ecosystem. The system was implemented using a TurtleBot3 Burger operating in a Gazebo simulation environment, while an NVIDIA Jetson Orin Nano served as the primary edge-computing platform responsible for language model inference, mission management, performance evaluation, and data logging. The framework combines natural language understanding, LiDAR-based perception, navigation control, recovery behaviours, and benchmarking capabilities to provide an intelligent robotic inspection platform. Figure 1 illustrates the overall mission execution flowchart of the proposed OpenClaw framework.

Figure 1. Overall mission execution flowchart of the proposed OpenClaw framework.

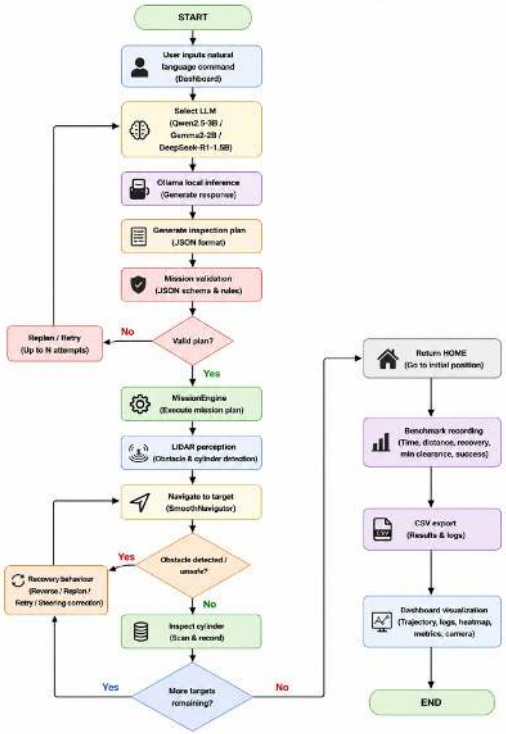

Figure 1. Overall mission execution flowchart of the proposed OpenClaw framework.

The inspection mission begins when a user provides a natural language instruction through the OpenClaw dashboard interface. Commands requesting inspection of one or multiple cylindrical targets are transmitted to Ollama, where one of the selected language models, namely Qwen2.5-3B, Gemma2-2B, or DeepSeek-R1-1.5B, generates a structured inspection plan. The generated response is subsequently validated to ensure that the mission sequence satisfies predefined execution requirements before being forwarded to the MissionEngine for task execution. Invalid responses trigger replanning procedures and retry mechanisms to improve mission reliability. During mission execution, LiDAR measurements continuously provide information regarding obstacle proximity and target locations within the hexagonal inspection arena, while odometry data are utilized to estimate robot displacement, travelled distance, and trajectory progression. The navigation subsystem generates velocity commands according to the relative position between the robot and the inspection target, where steering corrections are continuously applied to improve trajectory smoothness and minimize oscillatory behaviour. Once the target is reached, a short inspection routine is performed before proceeding to the next assigned location. The software organization of the developed framework is illustrated in Figure 2.

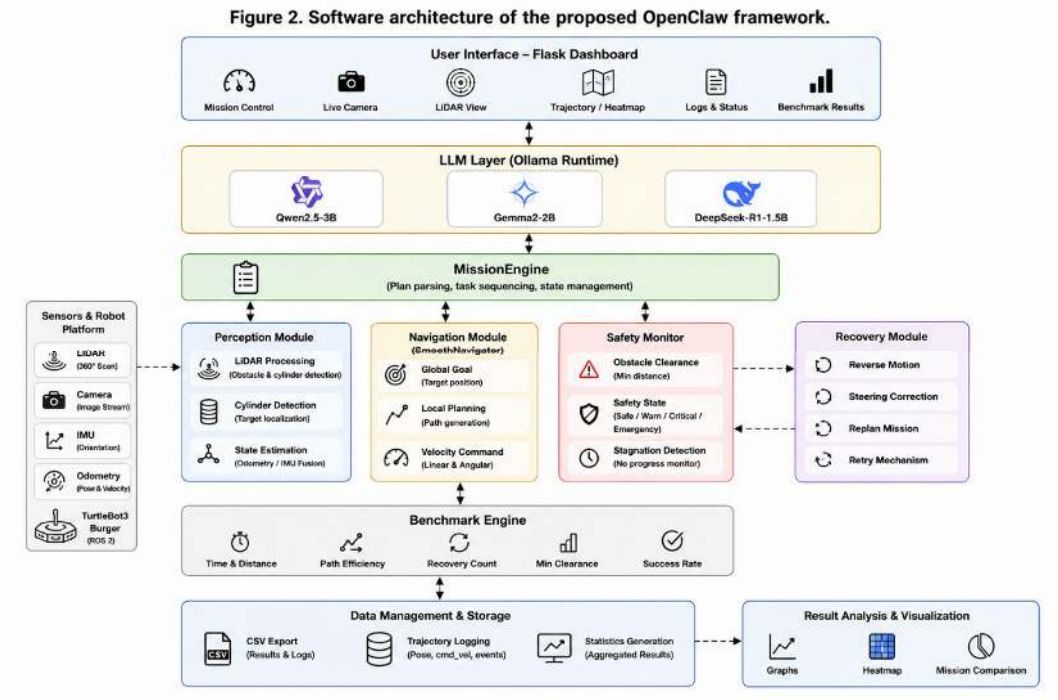

Figure 2. Software architecture of the proposed OpenClaw framework.

To ensure safe and reliable operation, the framework incorporates a safety monitoring mechanism based on obstacle clearance measurements obtained from LiDAR sensors. Robot operating conditions are categorized into safe, warning, critical, and emergency states according to the minimum detected obstacle distance. Whenever navigation stagnation or abnormal movement behaviour is identified, manoeuvre-based recovery strategies are activated. These recovery behaviours include reverse motion, steering correction, replanning attempts, and retry procedures before resuming normal mission execution. Upon completion of all inspection objectives, the robot autonomously returns to its initial home position to ensure repeatable experimental conditions.

Benchmark automation was implemented to facilitate comparative evaluation of multiple language models under identical operating conditions. Prior to each benchmark run, the robot is reset to the home position to eliminate positional bias and maintain experiment consistency. A total of 120 inspection missions consisting of forty runs per language model were conducted. During each experiment, performance indicators including planning latency, mission completion time, navigation distance, path efficiency, recovery count, obstacle clearance, and mission success rate are collected and exported into comma-separated value (CSV) files for statistical analysis. To support system visualization and monitoring, a Flask-based dashboard was integrated into the framework. The dashboard provides real-time information regarding robot status, mission progress, benchmark results, trajectory visualization, system logs, LiDAR measurements, and live camera streams, enabling users to monitor inspection activities and compare language model performance through a centralized monitoring platform. Table 1 summarizes the hardware and software configuration utilized in the developed OpenClaw framework.

Component	Specification
Mobile Robot	TurtleBot3 Burger
Edge Computing Device	NVIDIA Jetson Orin Nano
Operating System	Ubuntu 22.04
Middleware	ROS 2 Humble
Simulator	Gazebo Classic
Runtime Environment	Ollama
Language Models	Qwen2.5-3B, Gemma2-2B and DeepSeek-R1-1.5B
Sensors	LiDAR, Camera, IMU and Odometry
Dashboard	Flask-based OpenClaw Interface
Benchmark Runs	120 (40 runs per model)

Table 1. Hardware and Software Specifications

The developed methodology provides a practical framework for evaluating lightweight language models in autonomous inspection tasks while maintaining safe navigation performance, repeatable benchmark conditions, and real-time monitoring capabilities. The proposed framework also enables systematic comparison of multiple language models under identical operating environments, thereby supporting the selection of suitable language models for future edge-based intelligent robotic applications.

Result

The developed OpenClaw framework was evaluated using three lightweight Large Language Models, namely Qwen2.5-3B, Gemma2-2B, and DeepSeek-R1-1.5B. To ensure a fair comparison, each model was subjected to forty autonomous inspection missions under identical operating conditions, resulting in a total of 120 benchmark runs. Prior to each experiment, the robot was automatically returned to the home position to eliminate positional bias and maintain repeatability throughout the evaluation process.

The benchmark evaluation focused on planning latency, mission completion capability, navigation efficiency, recovery behaviour, and overall deployment readiness. Figure 3 presents the average planning latency obtained from the benchmark experiments. Lower planning latency indicates faster task generation and improved responsiveness during autonomous mission execution. Among the evaluated models, Qwen2.5-3B demonstrated the lowest average planning latency, while Gemma2-2B exhibited competitive performance with stable mission execution behaviour. DeepSeek-R1-1.5B achieved satisfactory task completion capability but required comparatively longer planning times under several inspection scenarios. The observed differences indicate that although all evaluated language models are capable of producing executable inspection plans, their suitability for

deployment in edge-based robotic applications may vary according to computational requirements and response characteristics.

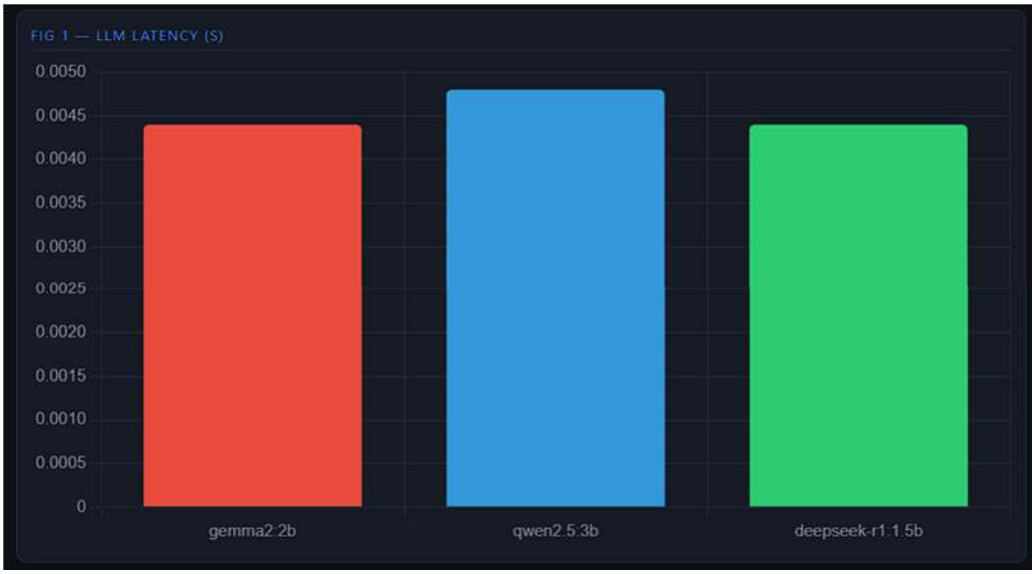

Figure 3. Comparative planning latency of Qwen2.5-3B, Gemma2-2B, and DeepSeek-R1-1.5B obtained from 120 autonomous inspection missions.

Besides planning latency, navigation performance was analysed based on travelled distance, path efficiency, recovery count, and mission success rate. Figure 4 illustrates an example autonomous inspection trajectory generated by the OpenClaw framework within the hexagonal inspection arena. The recorded trajectory demonstrates that the robot was capable of navigating towards designated cylindrical targets, executing inspection routines, avoiding surrounding obstacles, and safely returning to the home position upon mission completion. The implementation of manoeuvre-based recovery strategies further improved mission robustness by reducing the probability of navigation failures caused by local stagnation or environmental constraints.

Table 2 summarizes the comparative benchmark results obtained from the experimental evaluation. The presented results indicate that all three language models successfully generated executable inspection plans under identical experimental conditions. However, differences were observed in planning latency, recovery frequency, mission completion consistency, and overall deployment readiness. These findings provide useful insights into the suitability of lightweight language models for autonomous robotic applications operating under edge-computing constraints.

Model	Latency (s)	Nav. Score (%)	Overall Score (%)
Gemma2-2B	0.004	72.80	75.25
Qwen2.5-3B	0.005	78.06	80.50
DeepSeek-R1-1.5B	0.004	78.16	80.65

Table 2. Comparative Benchmark Results of the Evaluated Language Models

Overall, the experimental results demonstrate the capability of the proposed OpenClaw framework to support systematic evaluation of multiple language models within autonomous inspection applications. The developed benchmark methodology provides a repeatable platform for analysing task planning performance, navigation behaviour, safety compliance, and deployment readiness of lightweight language models operating on edge-computing devices. The obtained results are

expected to assist future researchers and developers in selecting suitable language models for intelligent robotic systems requiring efficient operation under limited computational resources.

Conclusion

This paper presented OpenClaw, a latency-aware framework for integrating Large Language Models (LLMs) with autonomous navigation and intelligent task planning in ROS 2 mobile robots. The proposed framework successfully combines natural language understanding, LiDAR-based environmental perception, autonomous navigation, safety monitoring, recovery behaviours, benchmark automation, and dashboard-based visualization within a unified robotic platform operating on an NVIDIA Jetson Orin Nano edge-computing device.

Comparative evaluation was conducted using Qwen2.5-3B, Gemma2-2B, and DeepSeek-R1-1.5B under identical autonomous inspection scenarios. The developed benchmark framework enabled systematic assessment of planning latency, navigation efficiency, mission completion capability, recovery performance, and deployment readiness. The obtained results demonstrate the feasibility of deploying lightweight language models in autonomous robotic applications while maintaining reliable navigation behaviour and repeatable experimental conditions.

Overall, the OpenClaw framework provides a practical platform for evaluating modern language intelligence in edge-based robotic systems. Future work may focus on extending the framework towards real-world robot deployment, enhanced perception capabilities, and migration to higher-fidelity simulation environments such as NVIDIA Isaac Sim to further improve environmental realism and system robustness.

Acknowledgement

The authors would like to express their sincere appreciation to Assoc. Prof. Ir. Dr. Zool Hilmi Ismail from the Malaysia–Japan International Institute of Technology (MJIIT), Universiti Teknologi Malaysia, for his valuable guidance, technical advice, and continuous support throughout the completion of this Final Year Project. The authors also acknowledge the use of open-source technologies, including ROS 2, Gazebo, Ollama, and TurtleBot3, which have significantly contributed to the development and evaluation of the proposed OpenClaw framework.

References

[1] Das M, Hussain Z, Nawaz M, editors. Latency-Aware Benchmarking of Large Language Models for Natural-Language Robot Navigation in ROS 2. Sensors. 2026;26(2):608.

[2] Bian S, Zhang Y, Tian G, Miao Z, Wu EQ, Yang SX, Hua C, editors. Large Language Model-Based Task Planning for Service Robots: A Review. Intelligent Computing. 2025.

[3] Jeong H, Kim J, Lee S, editors. A Survey of Robot Intelligence with Large Language Models. Applied Sciences. 2024;14(19):8868.

[4] Authors, editors. Robot Planning with Large Language Models for Autonomous Robotic Systems. Nature Machine Intelligence. 2025.